Singapore Programming Languages Summit 2nd edition, Nov 24 2025

Reflections on Duality

Ding Feng Kyriel Mortel Abad

National University of Singapore

Advisor: Michael D. Adams, National University of Singapore

Table of Contents

I. Introduction to Duality

II. A Dual Language

III. Explorations

IV. Conclusion

V. Call to Action

Introduction of Duality

- A brief look at the sequent calculus

Natural Deduction

One output on the right side of the turnstile (asymmetric)

$$A_1, A_2, \ldots, A_n \vdash B$$

Sequent Calculus

Many-to-Many structure in a sequent (symmetric)

$$A_1, \ldots, A_n \vdash B_1, \ldots, B_k$$

Introduction of Duality

- What does duality mean, in computation?

Let's think about Curry-Howard Correspondence:

- **Functions** take in many arguments but always return one value, similar to a step in **Natural Deduction**:
  - **ND** - Lacks information on structure & behavior of proofs, awkward to model classical logic (Gentzen, 1935)
  - **λ-calculus** - Lacks of information on control-flow, awkward to represent continuations…

- What is the corresponding framework for **Sequent Calculus**?

Introduction of Duality

- What does duality mean, in computation? cont.

What is the corresponding framework for Sequent Calculus?

- $\bar{\lambda}\mu\tilde{\mu}$-calculus, Curien & Herbelin, 2000
- Dual calculus, P. Wadler, 2003
- …

Benefits?

- Explicit control flow manipulation
- First-class continuations
- Eagerness and Laziness in one calculus

Introduction of Duality

- Dual calculus in a nutshell

In dual calculus, we model computation as the interaction between **expression** and **continuation**, rather than function applications

Introduction of Duality

- Problems?

But…

- It is still quite underexplored, we know little about the sugars, pragmatics, type systems…

Interlude

- To solve the problem - **begin with implementing a dual language**
- *Existing: Theoretical, or as IR for other languages*
- **Rationale:**
  - **Simple**, so that people can easily understand
  - **Powerful**, it should be able to demonstrate full power of duality
  - **Ergonomic**, people should be able write & read the program with minimal effort

A Dual Language

- What does a dual language looks like?

```
-- data
halt := Halt
zero := Z
one := S (Z)
two := S (S (Z))

-- codata
add := {
  (x, y, k) →
    x . { Z → y . k
        | S x' → add . (x', S y, k) } }

-- command
add . (one, two, halt)
```

Constructor(∧)

μ-Abstraction (∨)

Command/Cut (enables ¬)

A Dual Language

- Two core abstractions

- A constructor is similar to data constructors in traditional way
- It produces **expression of data** and **continuation of codata**
- **Conjunction (∧)** in logic

```
halt := Halt
zero := Z
one := S (Z)
two := S (S (Z))
```

A Dual Language

- Two core abstractions, cont.

- μ-abstraction is a special abstraction **to observe structures**

- It is **continuation of data** and **produces codata expressions**

- **Disjunction (∨)** in logic

```
{ Z → y . k
| S x' → add . (x', S y, k) }
```

A Dual Language

- Command/Cut

- A pair of a expression and a continuation
- Models the "sequent" in sequent calculus
- **Every computation step can be abstracted as a command**
- This enables ¬

```
-- command
add . (one, two, halt)
```

A Dual Language

- Dynamics Demo

```
-- 1.
add . (one, two, halt)

-- 2.
{ (x, y, k) →
    x . { Z → y . k
        | S x' → add . (x', S y, k) } } . (S(Z), S(S(Z)), halt)
```

Next

A Dual Language

- Dynamics Demo

```
-- 3. pattern matched, continue with inner command
S(Z) . { Z → S(S(Z)) . halt
       | S x' → add . (x', S(S(S(Z))), halt) }

-- 4. matched to S x', continue with inner command
add . (Z, S(S(S(Z))), halt)
```

A Dual Language

- Dynamics Demo

```
-- 5.
Z . { Z → S(S(S(Z))) . halt
    | S x' → add . (x', S(S(S(S(Z)))), halt) }

-- 6. matched to Z, continue with inner command
S(S(S(Z))) . halt

-- 7. halt with S(S(S(Z)))
```

A Dual Language

- The first duo: expression and continuation

- In dual calculus, <u>expressions and continuations are treated equally</u>
- We can “continuationalize (adding negation)” an expression by switching it to the other side of command, and vice versa
- Example:

```
-- Expression tuple and continuation addition-function
(1, 2, halt) . add

-- Expression addition-function and continuation tuple
add . (1, 2, halt)

-- Both evaluate to < 3 . halt >
```

A Dual Language

- Data and co-data, natively as duo

```
-- a stream of infinite 1s, example of codata
ones := { Head k_h → 1 . k_h
        | Tail k_t → ones . k_t }

-- pair and co-pair
pair := Pair 1 2

co-pair := { Fst k → 1 . k | Snd k → 2 . k } -- lazy
```

A Dual Language

- One more duo: monads & co-monads

```
-- option monad version 1
-- implementation as data type
fn return_d v k := (Some v) . k;

fn bind_d opt f k :=
  opt . { None → None . k
        | Some v → f(v) . k };

-- option monad version 2
-- implementation as codata type
fn return_c v k :=
  { (some_k, none_k) → v . some_k } . k;

fn none_c k :=
  { (some_k, none_k) → Empty . none_k } . k;

fn bind_c opt f k :=
  { (some_k, none_k) →
    opt . ( { v → f(v) . (some_k, none_k) } , none_k)
  } . k;
```

```
-- option comonad version 1
-- implementation as data type
fn extract_d opt k :=
  opt . { None → None_error . halt
        | Some v → v . k };

fn extend_d opt f k :=
  f(opt) . { v → Some v . k };

-- option comonad version 2
-- implementation as codata type
fn extract_c opt k :=
  opt . ( { v → v . k }, {_ → None_error . halt} )

fn extend_c opt f k :=
  { (some_k, none_k) → f(opt) . (some_k, none_k) } . k;
```

A Dual Language

- One more duo: monads & co-monads

```
-- option monad version 1
-- implementation as data type
fn return_d v k := (Some v) . k;

fn bind_d opt f k :=
  opt

-- opt
-- imp
fn ret
  { (s

fn none_c k :=
  { (some_k, none_k) → Empty . none_k } . k;

fn bind_c opt f k :=
  { (some_k, none_k) →
    opt . ( { v → f(v) . (some_k, none_k) } , none_k)
  } . k;
```

```
-- option comonad version 1
-- implementation as data type
fn extract_d opt k :=
  opt . { None → None_error . halt

-- implementation as codata type
fn extract_c opt k :=
  opt . ( { v → v . k }, {_ → None_error . halt} )

fn extend_c opt f k :=
  { (some_k, none_k) → f(opt) . (some_k, none_k) } . k;
```

Demonstrate the **2 for 1 ability** of dual calculus,

people <u>get 2 concepts by inventing one of them</u>

Explorations

- Syntax sugars

- Unsugared dual language is extremely painful to read or write
  - Deeply nested code and explicit command structure
- No higher level abstractions for human reader or writer
- We implemented a lot of sugars attempting to solve the ergonomics issue

Explorations

- Syntax sugars - Model FP easier

```
quick_sort := { (xs, k) →
  xs . { Nil → Nil . k
       | List:: pivot xs' →
          list_split . (lt, xs', pivot, {
            (smaller, greater) →
              quick_sort . (smaller, {
                sorted_s →
                  quick_sort . (greater, {
                    sorted_g →
                      list_append . (sorted_s, (List:: pivot sorted_g)) . k }) }) }) } }
```

Before

```
fn quick_sort xs k :=
  match xs with
    Nil → Nil . k
  | List:: pivot xs' →
      do
        (smaller, greater) ← list_split(lt, xs', pivot),
        sorted_s ← quick_sort(smaller),
        sorted_g ← quick_sort(greater)
      then list_append(sorted_s, (List:: pivot sorted_g)) . k
```

After

Explorations

- Library & Sample Programs

<u>In order to find the pragmatics/idioms of dual programming:</u>

- Write a bunch of sample programs in language we built
  - Leetcode questions
  - Programs from Wikipedia articles, especially those about continuations
- Write libraries for booleans, natural numbers, list…

Works In Progress

- What we are actively working on

- Modeling delimited continuation
  - We already have a prototype that can model shift/reset
- Type system
  - Likely still “two for one”, e.g. if we have ∀ types we get ∃
- Figuring out more idioms of dual programming
  - Instead of fit into existing paradigms
- Visualization e.g. https://pythontutor.com/
- Discovery of applications

Conclusion

- Dual calculus brings **continuation onto the front stage**
- The theoretical beauty of the dual calculus can be realized in a **simple & ergonomic programming language**
- A simple & ergonomic language **makes exploration easier**
- Dual calculus **can be readable and writable**

People & Call to Action

Ding Feng

dingfeng@u.nus.edu

Year 3 Undergraduate

Kyriel Mortel Abad

kyriel@u.nus.edu

Year 4 Undergraduate

Michael D. Adams

adamsmd@nus.edu.sg

Assistant Professor

Halt

- The continuation of everything

Link to Repo

Halt = Q & A

References

1. Curien, P.-L., & Herbelin, H. (2000). The duality of computation. SIGPLAN Not., 35(9), 233– 243. https://doi.org/10.1145/357766.351262
2. Downen, P., & Ariola, Z. M. (2018). A tutorial on computational classical logic and the sequent calculus. Journal of Functional Programming, 28, e3. https://doi.org/10.1017/S0956796818000023
3. Gentzen, G. (1935a). Untersuchungen über das logische schließen. I. Mathematische Zeitschrift, 39(1), 176–210. https://doi.org/10.1007/BF01201353
4. Gentzen, G. (1935b). Untersuchungen über das logische schließen. ii. Mathematische Zeitschrift, 39(1), 405–431. https://doi.org/10.1007/BF01201363
5. Wadler, P. (2003). Call-by-value is dual to call-by-name. SIGPLAN Not., 38(9), 189–201. https://doi.org/10.1145/944746.944723